Bubble Sort

AlgoExpert · Easy · Sorting

The problem

- Repeatedly swap adjacent out-of-order pairs; stop early when a pass makes no swaps.

Intuition

- Repeatedly swap neighbours that are out of order; each pass bubbles the largest remaining item to the end.

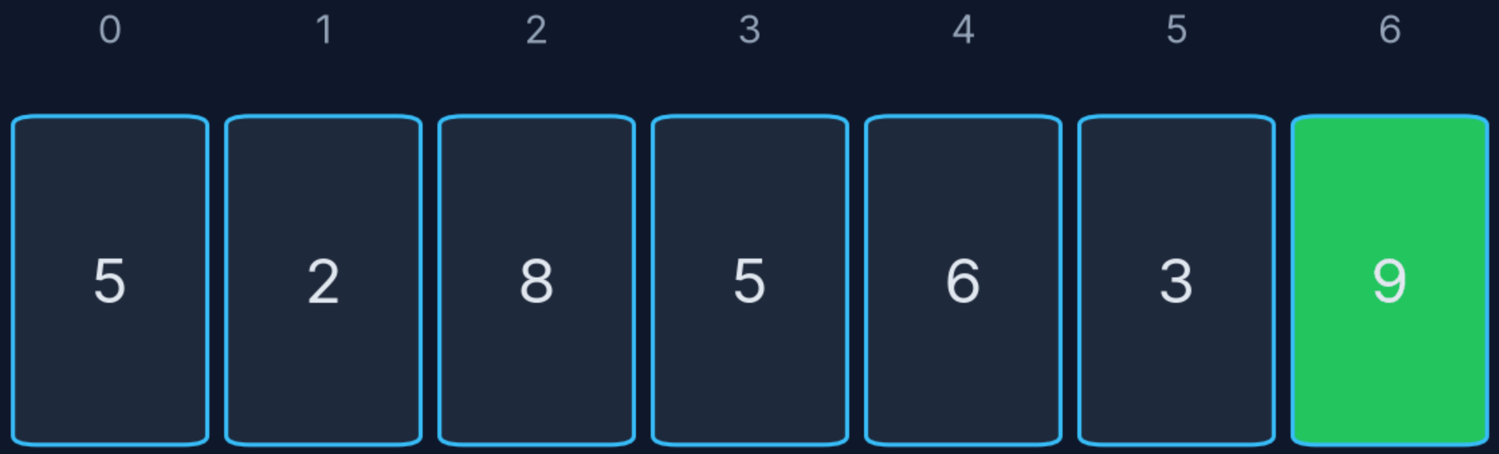
Brute force vs optimal

- Bubble sort is $O(n^2)$ — fine for learning, not for production
- Better: merge/quick/heap sort → $O(n \log n)$
- Early exit makes sorted input $O(n)$

Approach

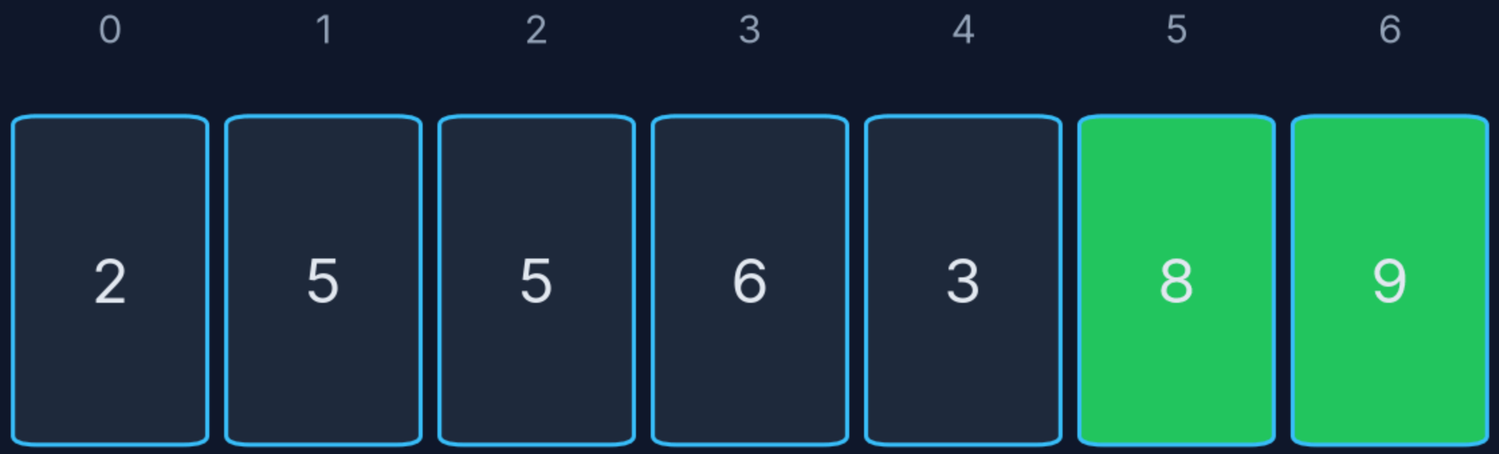
- 1. Loop until a pass makes no swaps
- 2. Swap adjacent pairs that are out of order
- 3. Shrink the range each pass

Worked example



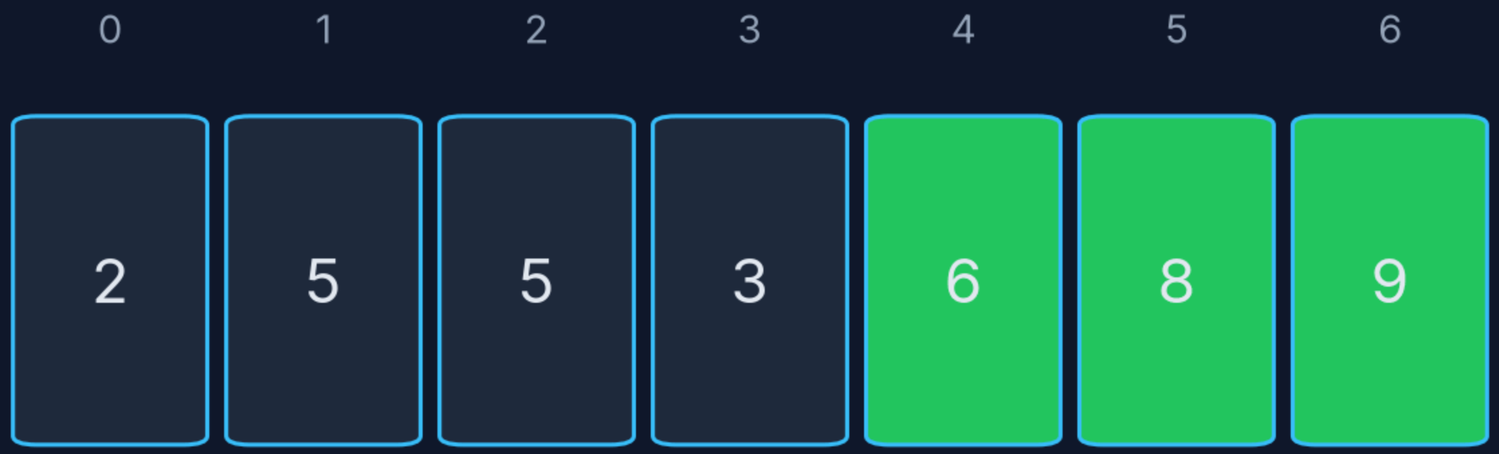
After pass 1: 9 bubbled to index 6

Worked example



After pass 2: 8 bubbled to index 5

Worked example



After pass 3: 6 bubbled to index 4

Worked example



After pass 5: 5 bubbled to index 2

Worked example



After pass 6: 3 bubbled to index 1

Java solution

```java
public class BubbleSort {
    public static int[] bubbleSort(int[] a) {
        boolean swapped = true;
        for (int end = a.length - 1; swapped && end > 0; end--) {
            swapped = false;
            for (int i = 0; i < end; i++)
                if (a[i] > a[i + 1]) { int t = a[i]; a[i] = a[i + 1]; a[i + 1] = t; swapped = true; }
        }
        return a;
    }
}
```

Complexity

- Time: O(n^2) worst, O(n) best
- Space: O(1)

Edge cases & pitfalls

- Shrink the inner range each pass
- Stop when a pass makes no swaps
- Stable: equal elements keep order

Interview tips & follow-ups

- Know the complexity of all simple sorts
- Follow-up: why is it stable? how many swaps = inversions?

Recap

- Repeatedly swap neighbours that are out of order; each pass bubbles the largest remaining item to the end.
- Complexity: O(n^2) worst, O(n) best